

Logic, Propositions and Logical Connectives

A 1.5-2 hour lecture combining formal logic, Boolean reasoning, CS applications and Python.

- Recommended teaching duration: approximately 1.5-2 hours including board work, worked examples, Python demonstrations and student discussion.
- Teaching approach: concept → notation → worked example → CS interpretation → Python verification → student practice.

Lecture Roadmap

- Concept introduction and terminology
- Detailed definitions and notation
- Worked mathematical examples
- CS case studies with complete solutions
- Python implementation and verification
- Extended question bank
- 2-3 unsolved student practice questions at the end

1. Introduction to Logic

Logic is the study of principles used to distinguish valid reasoning from invalid reasoning. In computer science, logic is used to specify requirements, design conditions, test software, construct digital circuits, query databases, and reason about program behavior.

Discrete mathematics treats objects and processes in separate, identifiable units. Logical statements are especially important because a computer ultimately evaluates conditions as Boolean values.

2. Statements / Propositions

A proposition is a declarative sentence that is either true or false, but not both. We normally represent propositions by p , q , r , and so on.

Examples: “8 is even” is true; “11 is divisible by 2” is false. Commands, questions and incomplete expressions are not propositions. For example, “Close the program” is a command, while “ $x > 5$ ” has no fixed truth value until x is specified.

Context can turn an expression into a proposition. If $x=3$ is already known, then $x>5$ is a false statement.

3. Simple and Compound Statements

A simple proposition cannot be divided into smaller propositions using logical connectives. A compound proposition is built from one or more propositions.

Example: p = “The server is online” and q = “The database is reachable.” Then $p \wedge q$ means both conditions hold.

4. Logical Connectives

Negation: $\neg p$ means “not p.” Conjunction: $p \wedge q$ means “p and q.” Disjunction: $p \vee q$ means “p or q,” where inclusive OR permits both. Implication $p \rightarrow q$ means “if p, then q.”

Biconditional $p \leftrightarrow q$ means “p if and only if q.”

Students should learn both the English interpretation and the truth condition of each connective. The implication is particularly important: it is false only when the premise is true and the conclusion is false.

5. Translating English into Symbols

“The user is authenticated and the account is active” $\rightarrow a \wedge b$.

“The server is unavailable or the network is disconnected” $\rightarrow s \vee n$.

“If the file exists, then it can be opened” $\rightarrow e \rightarrow o$.

“A request is accepted if and only if its token is valid” $\rightarrow \text{accepted} \leftrightarrow \text{valid_token}$.

6. Operator Precedence

When several connectives appear, use parentheses. A useful conventional order is negation first, then conjunction, then disjunction, then implication and biconditional. In programming, parentheses are strongly recommended even when precedence is known.

7. Python Connection

Python's not, and, and or correspond naturally to logical connectives. Comparisons such as $x > 10$, $\text{username} == \text{'admin'}$, and $\text{score} \geq 50$ produce Boolean values.

8. Worked Examples

Example A: Let $p = \text{True}$ and $q = \text{False}$. Then $\neg p = \text{False}$, $p \wedge q = \text{False}$, $p \vee q = \text{True}$. Since $p \rightarrow q$ has a true premise and false conclusion, it is False.

Example B: Let p mean “student submitted” and q mean “file uploaded.” The confirmation rule $p \wedge q$ is true only when both events occur.

Example C: Let p mean “admin” and q mean “active account.” $\text{Access} = p \wedge q$. An administrator with a disabled account is therefore denied access.

9. CS Case Study — Authentication System

Scenario: A web application requires a valid username, a correct password, and an active account. Define u , p , a accordingly.

Solution: $\text{Login} = u \wedge p \wedge a$. If $u = \text{True}$, $p = \text{True}$, $a = \text{False}$, $\text{login} = \text{False}$. Python can evaluate the same rule.

10. CS Case Study — Monitoring System

Scenario: An urgent alert is generated when a server is down OR CPU usage exceeds 90%. A warning is generated when CPU exceeds 75% AND the server is not down.

Solution: $\text{urgent} = d \vee c90$. $\text{warning} = c75 \wedge \neg d$. If $d = \text{False}$, $c90 = \text{False}$, $c75 = \text{True}$, the result is $\text{urgent} = \text{False}$ and $\text{warning} = \text{True}$.

Python Laboratory

Python Activity 1

```
# Boolean logic in Python
valid_user = True
valid_password = True
active_account = False

login = valid_user and valid_password and active_account
print("Login allowed:", login)
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Python Activity 2

```
# Translating an implication-style rule
file_exists = True
file_readable = True

# p -> q is equivalent to (not p) or q
rule_satisfied = (not file_exists) or file_readable
print(rule_satisfied)
```

Ask students to predict the output before executing the code, then change at least one input and explain the new result.

Extended Question Bank — Instructor-Led Practice

The following questions are designed to provide enough board/classroom practice for a 1.5–2 hour session. They cover the same topic areas as the relevant VU MTH202 sequence, but are written as fresh exercises rather than reproducing the handout wording.

1. Classify as proposition or not: “12 is even”, “Open the browser”, “ $x+4=9$ ”, “Pakistan is in Asia.”
2. Write symbols for: server is online AND database is reachable.
3. Write symbols for: user is admin OR user is instructor.
4. Translate: if a file is encrypted, then a valid key is required.
5. Translate: access is granted if and only if the account is active.
6. Find the truth value of $\neg p$, $p \wedge q$, $p \vee q$ and $p \rightarrow q$ for $p = \text{True}$, $q = \text{False}$.
7. Give a CS example of a proposition that is true and one that is false.
8. Explain why $x > 5$ is not a proposition when x is unspecified.
9. Write a Python condition for: score is at least 50 AND attendance is at least 75%.
10. Write a Python condition for: admin OR instructor, but account must be active.
11. Create a Python program that asks for three Boolean inputs and evaluates a login rule.

12. Explain the difference between inclusive OR and an exclusive choice in a software interface.

End-of-Lecture Student Practice — No Solutions

1. A university portal grants access if the user is authenticated AND (admin OR instructor), but never if the account is suspended. Translate and implement the rule.
2. Construct a truth table for a two-factor authentication policy and identify when access is denied.
3. Write a Python program that evaluates the policy for all Boolean combinations.

Quick Recap

Logic converts conditions into precise Boolean expressions.